

Recorrido del código

Qué hace cada parte de la interfaz gráfica

Playa de Estacionamiento — Grupo 13

Archivo: **main.py**

Este documento recorre el **código real** de `main.py`, la ventana del programa. Está hecha con una herramienta llamada **PyQt5** (una caja de piezas para armar ventanas: botones, tablas, pestañas). Los nombres en **este color** son nombres reales del código.

1. La idea general del archivo

Este archivo **no hace cuentas**: las pide y las muestra. Importa el motor con `import simulacion as sim` y se ocupa de tres cosas: **(1)** dibujar la ventana, **(2)** tomar lo que el usuario carga y mandárselo al motor, y **(3)** mostrar los resultados en tablas con colores. El programa arranca al final del archivo, en el bloque `if __name__ == "__main__":`, que crea la ventana y la muestra.

2. Piezas de presentación (cómo se ven los datos)

`fmt(v)` — dar formato a cada celda

Función chiquita que decide cómo se ve cada valor: convierte `None` en un guión, los verdadero/falso en "Sí/No", y recorta los decimales largos. Es puro maquillaje de los números.

`construir_columnas(n)` — definir las columnas

Arma la lista de todas las columnas de la tabla grande, agrupadas por tema ("Llegada Auto", "Datos Auto", "Cobro", un grupo por cada lugar, etc.). Para cada columna guarda un grupo, un título y una **instrucción de dónde sacar el dato** de cada fila. Como la playa puede tener 8 o 10 lugares, las columnas se generan según `n`.

`COLOR_ESTADO` — la paleta de estados

Es una tabla que asocia cada estado con un color: por ejemplo Libre en verde, ocupado en naranja, playa llena en rojo. Después la tabla usa esto para pintar las celdas y comunicar de un vistazo.

3. Las tablas en pantalla

Una tabla en PyQt5 separa **los datos** de **cómo se dibujan**. Por eso hay varias clases:

Clase	Su rol
<code>TablaModel</code>	Es el "depósito de datos" de la tabla. Sabe cuántas filas y columnas hay y, cuando la pantalla le pregunta por una celda, le entrega el valor ya formateado y su color.
<code>GroupedHeader</code>	Dibuja el encabezado de dos niveles : arriba el nombre del grupo (con su color) y abajo el título de cada columna.
<code>SepDelegate</code>	Dibuja una línea vertical al final de cada grupo de columnas, para separarlos visualmente.
<code>VectorView</code>	Es la tabla en sí. Su truco principal: mantiene las primeras 3 columnas congeladas (número, reloj y evento) para que no se pierdan al desplazarse a la derecha, como en Excel.

Toda esta sección es solo **presentación**: no cambia ningún resultado, solo hace que la tabla —que es muy ancha— sea legible.

4. `SimWorker` — simular sin congelar la ventana

Si el programa hiciera la simulación "en el mismo lugar" donde dibuja la ventana, la ventana se quedaría tildada hasta terminar. Para evitarlo, `SimWorker` corre la simulación **en segundo plano** (un hilo aparte). Cuando termina, avisa con una señal:

```
class SimWorker(QtCore.QThread):
    terminado = QtCore.pyqtSignal(object, object, object) # éxito
    fallo      = QtCore.pyqtSignal(str)                  # hubo un error

    def run(self):
        try:
            ... # configura el motor y llama a sim.simular(...)
            self.terminado.emit(filas, stats, ...)
        except Exception as e:
            self.fallo.emit(str(e))
```

Tiene dos señales: `terminado` (cuando todo salió bien, con los resultados) y `fallo` (cuando el motor lanza un error). El `try / except` **atrapa cualquier problema** y lo convierte en un aviso, para que la ventana muestre el motivo en vez de cerrarse de golpe.

5. `Main` — la ventana principal

Es la clase central: arma la ventana entera y conecta los botones con las acciones.

`_panel_parametros()` — las casillas de carga

Crea todas las casillas donde se cargan los datos (lugares, media, precios, porcentajes, etc.) y los botones ■ `Simular`, `Re-aplicar i/j` y `Copiar a Excel`. Cada casilla queda guardada en un diccionario `self.inp` para poder leerla después.

`_recoger_parametros()` y `_num(...)` — validar lo que se carga

Antes de simular, el programa **revisa que todo lo cargado tenga sentido**. `_num(...)` lee una casilla, verifica que sea un número y que respete un mínimo (por ejemplo, los lugares no pueden ser menos de 1, ni la media ser 0). `_recoger_parametros()` usa esa función en cada campo, junta todos los errores y además controla que los porcentajes sumen 100.

Si algo está mal, no simula: **devuelve la lista de errores** para mostrarlos. Así un dato imposible (una letra, un negativo, porcentajes que no suman) nunca rompe el programa.

`on_simular()` — el botón de arranque

Es lo que se ejecuta al apretar ■ `Simular`. Su lógica es directa:

```
def on_simular(self):
    params, errores = self._recoger_parametros()
    if errores:
        QMessageBox.warning(self, "Revisá los parámetros", ...) # avisa y NO simula
        return
    ...
    self.worker = SimWorker(params)
    self.worker.terminado.connect(self.on_listo)
    self.worker.fallo.connect(self.on_fallo)
    self.worker.start()
```

Primero valida. Si hay errores, muestra un cartel y corta. Si está todo bien, lanza el `SimWorker` en segundo plano y le dice a quién avisar cuando termine: `on_listo` si salió bien, `on_fallo` si hubo un error.

`on_listo(...)` y `on_fallo(...)` — recibir el resultado

`on_listo` recibe las filas y las estadísticas, llena las tablas (incluida la de auditoría Euler) y muestra abajo la recaudación y el porcentaje de utilización. `on_fallo` muestra un cartel rojo con el motivo del error y vuelve a habilitar el botón.

Acciones cómodas

- `aplicar_filtro()`: muestra solo un tramo de filas (desde cierta hora, o una cantidad máxima), porque una corrida puede generar muchísimas.
- `on_celda_click(...)`: si se hace clic en el tiempo de cobro de un auto, salta al detalle de ese cálculo en la pestaña de Euler.
- `on_resumen_click(...)` / `_ir_a_euler(...)`: muestran, a la derecha, la tabla de pasos del cálculo del auto elegido.
- `on_copiar()`: copia la tabla visible al portapapeles para pegarla en Excel.

6. En resumen

Bloque del código	Para qué está
Piezas de presentación (<code>fmt</code> , <code>construir_columnas</code> , <code>COLOR_ESTADO</code>)	Definen cómo se ven los datos: formato, columnas y colores.
Clases de tabla (<code>TablaModel</code> , <code>VectorView</code> ...)	Muestran la tabla ancha de forma legible, con encabezados agrupados y columnas congeladas.
<code>SimWorker</code>	Corre la simulación en segundo plano y avisa por señales si terminó o falló.
<code>Main</code> (la ventana)	Arma la pantalla, valida lo que se carga , lanza la simulación y muestra los resultados.

Para llevarse: `main.py` es la cara visible del programa. Su parte más importante para la robustez es `_recoger_parametros()`, que revisa los datos antes de simular, y el `try/except` del `SimWorker`, que convierte cualquier error del motor en un aviso claro en vez de un cierre inesperado.